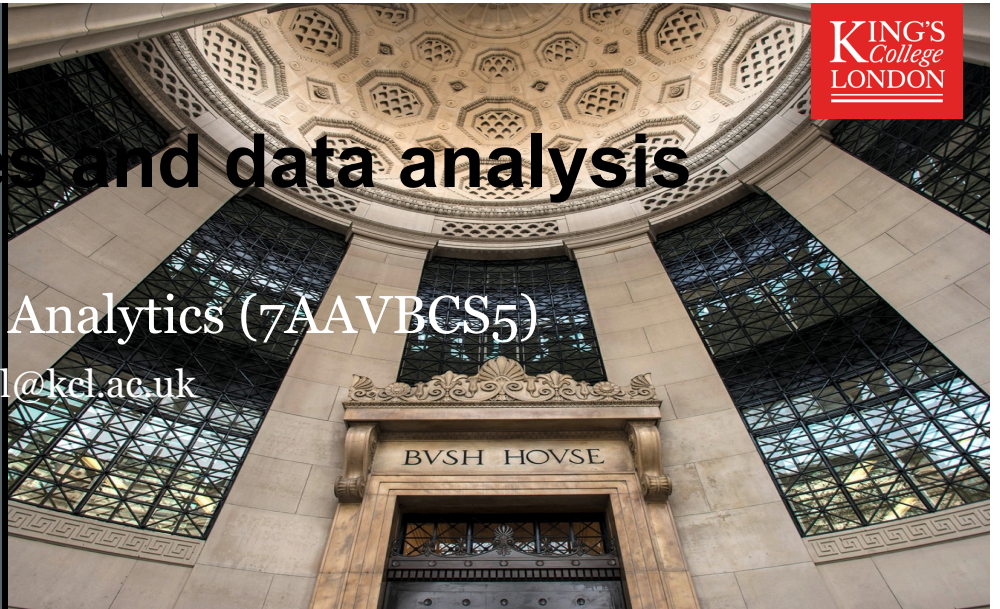


Variable types and data analysis

Social and Cultural Analytics (7AAVBCS5)

Dr Mark J. Hill · mark.j.hill@kcl.ac.uk

2026-10-26



Lecture Plan

- **Part I: Categorical Data**

- Nominal and ordinal
- Python's categorical type
- Ordered vs unordered
- Converting between types

- **Part II: Tables**

- Frequency tables
- Cross-tabulation
- Counts vs percentages

- **Part III: Grouping**

- `groupby` for numbers

- **Part IV: Live Coding**

Key Learning Outcomes

By the end of this week you will be able to:

- Understand and work with categorical data
- Create and interpret frequency tables
- Cross-tabulate two variables

- Choose between counts and percentages, and justify it
- Summarise a numeric column by group
- Apply these techniques to real datasets

Review: what we've learned so far

- Types, variables, lists, arrays, dictionaries
 - Comparisons and `if/elif/else`
 - Data frames: building, inspecting, selecting, filtering
 - `.loc` and `.iloc`
 - Reading and writing CSV
 - Missing data: finding it, quantifying it, deciding what to do about it
 - Whether missingness is **patterned**, and what that means
- This week:** the categories in your data, and counting things properly

Part I: Categorical Data

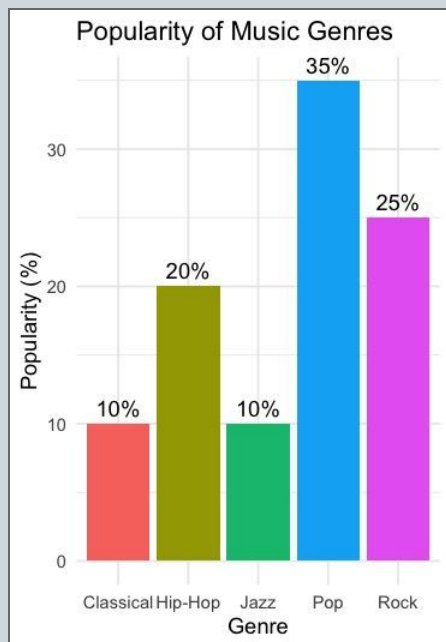
Categorical data: Nominal

Variables that represent categories or groups.

Nominal categorical variables represent qualitative differences between things. For example:

- Genre of music (rock, hiphop, classical)
- Types of phone (iPhone, Android)

These are things which **cannot be measured as numbers**.



Categorical data: Ordinal

Variables that represent categories or groups.

Ordinal categorical variables represent an order, ranking, or logical sequence. For example:

- Education level (high school, bachelor, master's, PhD)
- Subjective opinions via Likert scales (strongly agree, agree, neither agree nor disagree, disagree, strongly disagree)

Can, in some instances, be measured as numbers.

Python's way of handling categories

By default pandas stores categories as plain text.

Converting to a **category** does two extra things:

- Records the **set of possible values**, including ones that never appear
- Optionally records an **order**

```
1 import pandas as pd
2
3 colours = pd.Series(["red", "blue", "red", "green", "blue"]).astype("category")
4 print(colours.cat.categories)
```

```
Index(['blue', 'green', 'red'], dtype='str')
```

Ordered categories

```
1 scale = pd.CategoricalDtype(["Low", "Medium", "High"], ordered=True)
2 satisfaction = pd.Series(["Low", "High", "Medium", "Low", "Low", "High"]).astype(scale)
3
4 print(satisfaction.cat.categories)
5 print("is it ordered?", satisfaction.cat.ordered)
6 print(satisfaction.min(), "<", satisfaction.max())
```

```
Index(['Low', 'Medium', 'High'], dtype='str')
is it ordered? True
Low < High
```

Essential operations

R	Python
<code>is.factor(x)</code>	<code>x.dtype == "category"</code>
<code>levels(x)</code>	<code>x.cat.categories</code>
<code>nlevels(x)</code>	<code>len(x.cat.categories)</code>
<code>summary(x)</code>	<code>x.value_counts()</code>
<code>droplevels(x)</code>	<code>x.cat.remove_unused_categories()</code>
<code>factor(x, levels = ...)</code>	<code>x.cat.reorder_categories([...])</code>
<code>as.character(x)</code>	<code>x.astype(str)</code>

The trap: categories are stored as numbers

```
1 nums = pd.Series(["10", "20", "30"]).astype("category")
2 print("the codes: ", list(nums.cat.codes))
3 print("what you wanted:", list(nums.astype(str).astype(int)))
```

```
the codes: [0, 1, 2]
what you wanted: [10, 20, 30]
```

`.cat.codes` gives you the **internal position numbers**, not your values. Convert to text first, then to number.

The other trap: unused categories linger

```
1 subset = colours[colours != "green"]
2 print("categories still recorded:", list(subset.cat.categories))
3 print("after cleaning up:      ",
4       list(subset.cat.remove_unused_categories().cat.categories))
```

```
categories still recorded: ['blue', 'green', 'red']
after cleaning up:        ['blue', 'red']
```

Part II: Tables

One-way tables: frequency counts

```
1 museum = pd.read_csv("../data/museum_data.csv")
2 museum["region"].value_counts()
```

```
region
South America    37
West Africa      31
Pacific Islands  29
Southeast Asia   27
North Africa     26
Name: count, dtype: int64
```

- Counts how many times each category appears
- **Already sorted**, most common first
- Excludes missing values by default

Proportions instead of counts

```
1 (museum["region"].value_counts(normalize=True) * 100).round(1)
```

```
region
South America    24.7
West Africa      20.7
Pacific Islands  19.3
Southeast Asia   18.0
North Africa     17.3
Name: proportion, dtype: float64
```

Missing values are hidden by default

```
1 print(museum["acquisition_method"].value_counts())
2 print()
3 print(museum["acquisition_method"].value_counts(dropna=False))
```

```
acquisition_method
Purchase          56
Colonial Collection 29
Donation          22
Name: count, dtype: int64
```

```
acquisition_method
Purchase          56
NaN              43
Colonial Collection 29
Donation          22
Name: count, dtype: int64
```

The first table quietly omits 43 objects. Always look at the second one at least once.

Two-way tables: cross-tabulation

```
1 pd.crosstab(museum["region"], museum["acquisition_method"])
```

acquisition_method	Colonial Collection	Donation	Purchase
region			
North Africa	4	5	11
Pacific Islands	3	2	15
South America	9	7	9
Southeast Asia	6	6	9
West Africa	7	2	12

- First argument makes the rows, second the columns
- Each cell counts observations in that combination

With totals

```
1 pd.crosstab(museum["region"], museum["acquisition_method"], margins=True)
```

acquisition_method	Colonial Collection	Donation	Purchase	All
region				
North Africa	4	5	11	20
Pacific Islands	3	2	15	20
South America	9	7	9	25
Southeast Asia	6	6	9	21
West Africa	7	2	12	21
All	29	22	56	107

Counts can mislead

Raw counts are only comparable when the groups are the same size.

They almost never are.

```
1 papers = pd.read_csv("../data/BritishAndIrishNewspapersTitleList_20191118.csv",
2                       low_memory=False)
3 papers["digitised"] = papers["online_status"].notna()
4 main = papers[papers["country_of_publication"].isin(
5     ["England", "Scotland", "Wales", "Ireland", "Northern Ireland"])]
6
7 pd.crosstab(main["country_of_publication"], main["digitised"])
```

digitised	False	True
country_of_publication		
England	18840	1625
Ireland	899	151
Northern Ireland	367	48
Scotland	1593	185
Wales	962	57

Percentages, and the answer changes

```
1 (pd.crosstab(main["country_of_publication"], main["digitised"],
2             normalize="index") * 100).round(1)
```

digitised	False	True
country_of_publication		
England	92.1	7.9
Ireland	85.6	14.4
Northern Ireland	88.4	11.6
Scotland	89.6	10.4
Wales	94.4	5.6

Wales 5.6% · England 7.9% · Scotland 10.4% · N. Ireland 11.6% · Ireland 14.4%

Which margin?

Question	Argument	Each ... sums to 100%
What share of the whole table?	<code>normalize=True</code>	table
Within each row?	<code>normalize="index"</code>	row
Within each column?	<code>normalize="columns"</code>	column

“What percentage of Welsh titles are digitised?” and “what percentage of digitised titles are Welsh?” are **different questions** with different answers. Choose the margin that matches your question.

Tables are data frames

```
1 counts = museum["region"].value_counts()
2 print("the labels:", list(counts.index))
3 print("the numbers:", list(counts.values))
4
5 as_table = counts.reset_index()
6 as_table.columns = ["region", "count"]
7 as_table
```

```
the labels: ['South America', 'West Africa', 'Pacific Islands', 'Southeast Asia', 'North Africa']
the numbers: [np.int64(37), np.int64(31), np.int64(29), np.int64(27), np.int64(26)]
```

	region	count
0	South America	37
1	West Africa	31
2	Pacific Islands	29
3	Southeast Asia	27
4	North Africa	26

Part III: Grouping

Crosstabs count; groupby measures

A crosstab counts **two categories** against each other. But often you want a **number** summarised by a category.

```
1 museum.groupby("region")["valuation"].mean().round(0)
```

```
region
North Africa      28423.0
Pacific Islands   27370.0
South America     29583.0
Southeast Asia    27889.0
West Africa       27367.0
Name: valuation, dtype: float64
```

Several statistics at once

```
1 museum.groupby("region")["valuation"].agg(["count", "mean", "median", "std"]).round(0)
```

	count	mean	median	std
region				
North Africa	26	28423.0	32000.0	16712.0
Pacific Islands	27	27370.0	28000.0	13537.0
South America	36	29583.0	32000.0	15322.0
Southeast Asia	27	27889.0	32000.0	16334.0
West Africa	30	27367.0	29000.0	12394.0

Always keep `count` in the table. A mean over three objects is not a finding.

Grouping by two things

```
1 (main.groupby(["country_of_publication", "digitised"])["first_date_held"]
2   .agg(["count", "median"])
3   .head(6))
```

		count	median
country_of_publication	digitised		
England	False	18590	1935.0
	True	1613	1852.0
Ireland	False	885	1874.0
	True	151	1848.0
Northern Ireland	False	358	1935.0
	True	48	1853.0

Where this is going

Dimensions	Tool
one category	<code>value_counts()</code>
two categories	<code>pd.crosstab()</code>
category and a number	<code>groupby().agg()</code>
a number on its own	<code>describe()</code> , and histograms next week

Hill, “Contextual Analytics”, §5.4 — designing the analysis around an interaction

Part IV: Live Coding

Exercise 1: categorical data

Using the historic newspapers dataset:

1. Which columns are categorical rather than numeric?
2. Convert `country_of_publication` to a category. How many categories are there?
3. Some values are compounds like `England|Wales`. Is that one category or two?
4. Filter to one country, then look at the categories still recorded

Exercise 2: tables and cross-tabulation

1. One-way frequency table of `country_of_publication`
2. Sort it, and find the mode
3. Proportions and percentages
4. Two-way table: country against digitisation
5. Add margin totals
6. Row percentages — and say what changed
7. Column percentages — and say what different question they answer
8. Convert a table to a data frame

Exercise 3: grouping

1. Median `first_date_held` by country
2. Add a count column. Which groups are too small to trust?
3. Group by country **and** digitisation status
4. Are digitised titles systematically older or newer?

Activity: work with your data

Load your project dataset and answer:

- How many missing values in each column? What percentage?
- Which columns are categorical and which numeric?
- What strategy would you use for each column with gaps?
- Produce **one** crosstab or groupby that is relevant to your research question

Before next week

1. Complete `w05_worksheet.ipynb` in the workshop
2. Reading week is next — use it on your dataset
3. Read Hill, “Contextual Analytics”, §5.4

Note

The quiz after reading week, in **Week 7**, covers **this week's** material.

Extra: finding the mode, and counting categories

```
1 print("most common region:", museum["region"].value_counts().idxmax())  
2 print("how many distinct regions:", museum["region"].nunique())
```

```
most common region: South America  
how many distinct regions: 5
```



Speaker notes